CSE 213 — Software Engineering

Design Principles (SOLID) & Basics of Design Patterns

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Anindya Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — principle and pattern-basics questions
Lesson 1 — Encapsulate What Varies, Program to an Interface, Composition — Lec 1 sl. 2–9; cohesion & coupling
Lesson 2 — SRP and the Open/Closed Principle — Lec 1 sl. 10–17; OCP rewrite
Lesson 3 — LSP, ISP and DIP — Lec 1 sl. 18–31; ISP bad example
Lesson 4 — What a Design Pattern Is — Lec 2 sl. 1–10; GoF catalogue, essential elements
Quick Notes — Design Principles — revision + key diagrams

NOTE Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Lessons 1–3: 'Lec 1 - Design Principle.pdf' (32 slides). Lesson 4: 'Lec 2 - Basics of Design Pattern.pdf' (10 slides).

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2022-23 (CSE 307)	Q2(c) Example of the Interface Segregation Principle (8) [L3]
2018-19 (CSE 307)	Q3(c) Cohesion and coupling (8) [L1]
2017-18 (CSE 307)	Q7(c) Tight cohesion and loose coupling; influence on design (8) [L1]
2016-17 (CSE 307)	Q8(d) GraphicEditor.getArea by m_type: which principle is violated? Rewrite (6⅔) [L2]
2014-15 (CSE 307)	Q6(a) Structural vs behavioural patterns; classification criteria (10) [L4]
2012-13 (CSE 307)	Q4(b) Motivation of ISP with a bad example (14⅔) [L3]
2011-12 (CSE 307)	Q4(c) Four essential elements of a design pattern (8) [L4]
2010-11 (CSE 307)	Q3(c) Subclassing vs delegation: names; which deserves preference? (5) [L1]

EXAM FOCUS Small but steady: ISP (2022-23 Q2(c), 2012-13 Q4(b)), OCP rewrite (2016-17 Q8(d)), cohesion/coupling (2018-19, 2017-18), pattern basics (2014-15, 2011-12). Learn one clean example per SOLID letter — the ISP and OCP examples below are exam-ready. Every pattern answer in the other note files is also judged by these principles, so read this file first.

Lesson 1 — Encapsulate What Varies, Program to an Interface, Composition

Slides 2–9 (Lec 1 - Design Principle)

1. Encapsulate what varies (sl. 2–5)

Find the parts of the program that change, and separate them from the parts that stay the same. The change is then localised: you edit one method or one class, not many.

Method level (sl. 2–3)

```
// BEFORE: tax rules are mixed into the total computation
double getOrderTotal(Order order) {
    double total = 0;
    for (Item item : order.items) total += item.price * item.quantity;
    if (order.country.equals("US")) total += total * 0.07;   // US sales tax
    else if (order.country.equals("EU")) total += total * 0.20; // EU VAT
    return total;
}
// AFTER: the part that varies (tax rate) gets its own method
double getOrderTotal(Order order) {
    double total = 0;
    for (Item item : order.items) total += item.price * item.quantity;
    total += total * getTaxRate(order.country);
    return total;
}
double getTaxRate(String country) {
    if (country.equals("US")) return 0.07;
    if (country.equals("EU")) return 0.20;
    return 0;
}
```

Class level (sl. 4–5)

When the tax logic grows (state rules, product rules, Chinese tax …), move it out of Order into its own class TaxCalculator(getTaxRate(country, state, product), getUSTax, getEUTax, getChineseTax). Order now delegates; it has one job.

2. Program to an interface, not an implementation (sl. 6)

Depend on abstractions. Example: Cat.eat(Sausage) works only for sausages. Introduce interface Food (getNutrition()) and write Cat.eat(Food). Any new food can be added without touching Cat.

The slide warns: right after this change you feel no benefit and the code is more complex. Do it only where you expect an extension point.

3. Favour composition over inheritance (sl. 7–9)

- **Diamond problem (sl. 7):** if ClassA and ClassB both implement doSomething() and ClassC extends both, the compiler cannot decide which one ClassC inherits. Java therefore forbids multiple class inheritance.
- **Tight coupling (sl. 8):** Company creates Designer, Programmer, Tester directly — Company depends on every concrete employee class, so a new kind of company or employee forces edits in Company.
- **Fix (sl. 9):** abstract Company with abstract getEmployees(); GameDevCompany and OutsourcingCompany override it; Employee is an interface with doWork(). Company now depends only on Employee. (This is the Factory Method idea — Lesson 5.)

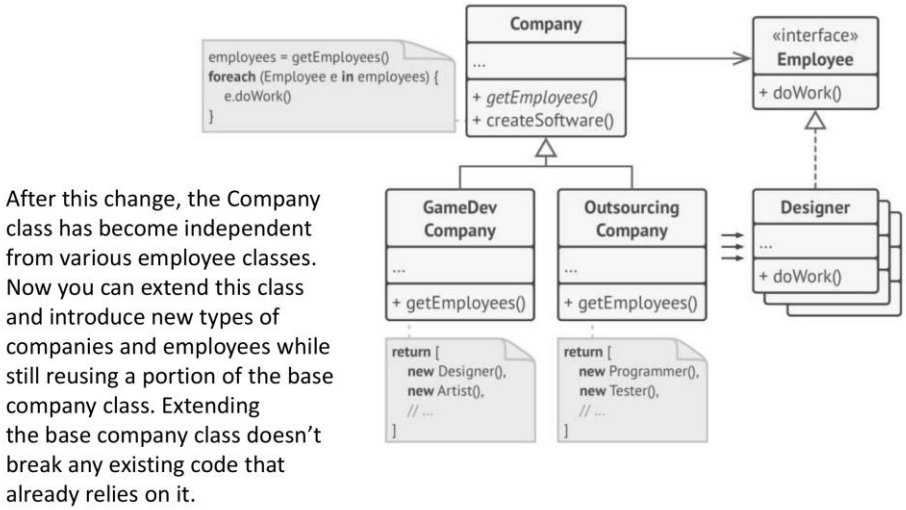


Figure 1.1 — Removing tight coupling: Company depends on the Employee interface (sl. 9)

4. Cohesion and coupling (textbook — asked 2018-19, 2017-18)

	Meaning	Want
Cohesion	How strongly the responsibilities inside one module/class belong together	High (tight) cohesion — one focused job
Coupling	How much one module depends on the internals of another	Low (loose) coupling — talk through narrow interfaces

EXAM-READY ANSWER Tight cohesion means each class does one well-defined job (SRP); loose coupling means classes interact only through abstract interfaces and know little about each other (DIP). Together they make a design easy to understand, test and change: a change stays inside one cohesive class, and because coupling is low it does not ripple into others. Example: Order with an embedded tax algorithm has low cohesion; moving tax to TaxCalculator behind an interface raises cohesion and lowers coupling.

5. Subclassing vs delegation (2010-11 Q3(c))

The two techniques in that question are **inheritance (subclassing)** and **delegation (object composition)**. Both fix the added behaviour at compile time. Composition/delegation should be preferred: it does not expose the parent's internals, avoids the diamond problem and deep hierarchies, and the delegate can be swapped at run time (this is how Strategy, Decorator and Bridge work).

Exam Approach

1. Name the principle in one line (quote the slide wording).
2. Show a 'before' with the smell (conditionals, concrete dependencies, fat class).
3. Show the 'after' with the extracted method/class or interface.
4. Finish with the benefit: what change is now local.

Practice — Lesson 1

(a) Actual — 2017-18 Q7(c). (b) Actual — 2018-19 Q3(c). (c) Actual — 2010-11 Q3(c).

(a) What is tight cohesion and loose coupling? How does it influence software design? (8)

(b) Discuss cohesion and coupling in brief in the context of software engineering. (8)

(c) Extending a system by subclassing vs by copying operations and delegating to another class: name the two techniques. Should one get preferential treatment? (5)

✓ Solution

(a) Use the EXAM-READY ANSWER in §4, then add an example: Company that creates Designer/Programmer/Tester itself is tightly coupled (sl. 8); after introducing Employee and getEmployees() it is loosely coupled (sl. 9). Influence: changes stay local, classes are reusable and testable in isolation, parallel development is easier.

(b) Cohesion = degree to which elements of a module belong together (aim high: functional cohesion). Coupling = degree of interdependence between modules (aim low: data coupling through parameters/interfaces, not content or common coupling). Good design = high cohesion + low coupling.

(c) Inheritance (subclassing) and delegation (composition). Prefer delegation: it keeps encapsulation, allows behaviour to be changed at run time, and avoids multiple-inheritance problems — 'favour composition over inheritance'.

Lesson 2 — SRP and the Open/Closed Principle

Slides 10–17 (Lec 1 - Design Principle)

1. SOLID (sl. 10)

Letter	Principle	One-line rule
S	Single Responsibility	A class should have just one reason to change.
O	Open/Closed	Classes should be open for extension but closed for modification.
L	Liskov Substitution	A subclass object must work wherever the parent object works.
I	Interface Segregation	Clients shouldn't be forced to depend on methods they do not use.
D	Dependency Inversion	High- and low-level classes both depend on abstractions.

Cost (sl. 10): applying them blindly can make a program more complicated than it should be.

2. Single Responsibility Principle (sl. 11)

Every class has a single responsibility, entirely encapsulated by the class. Example: Employee that also prints a time-sheet report has two reasons to change (employee data, report format). Move printing to TimeSheetReport.print(employee).

Benefits listed on the slide: less fragile code, low coupling, high cohesion, changeability, maintainability, testability and easier debugging.

3. Open/Closed Principle (sl. 12–17)

Once a class is complete, it is modified only to fix errors; new features come through new classes (sl. 13). Shipping example (sl. 12): Order.getShippingCost() with if (shipping == ground) … else if (air) … must be edited for every new method. Extract interface Shipping with Ground and Air implementations — a Strategy.

Worked example — AreaCalculator (sl. 14–17, C# on the slides, Java here)

```
// sl. 15 – every new shape forces an edit here (VIOLATES OCP)
double area(Object[] shapes) {
    double a = 0;
    for (Object s : shapes) {
        if (s instanceof Rectangle) { Rectangle r = (Rectangle) s; a += r.width * r.height; }
        else { Circle c = (Circle) s; a += c.radius * c.radius * Math.PI; }
    }
    return a;
}
// sl. 16-17 – closed for modification, open for extension
abstract class Shape { abstract double area(); }
class Rectangle extends Shape { double width, height; double area() { return width * height; }
}
class Circle    extends Shape { double radius;        double area() { return Math.PI * radius *
radius; } }
class Ellipse   extends Shape { double major, minor;  double area() { return Math.PI * major *
minor; } } // added later
class AreaCalculator {
    double area(Shape[] shapes) { double a = 0; for (Shape s : shapes) a += s.area(); return a;
}
}
```

MEMORIZE OCP smell = a type code or instanceof chain that must be edited for every new type. Cure = polymorphism: push each branch into an overriding method (refactoring 'Replace Conditional with Polymorphism').

Practice — Lesson 2

(a) Actual — 2016-17 Q8(d). (b) Constructed.

(a) Which class design principle does the following code violate? Rewrite the code to remove the violation.

(b) State SRP. Show a class that violates it and fix it.

```
class GraphicEditor {
    public float getArea(Shape s) {
        if (s.m_type == 1) return s.width * s.width;
        else if (s.m_type == 2) return 3.1416 * s.r * s.r;
    }
}
class Shape { int m_type; }
class Rectangle extends Shape { int width; Rectangle(int width) { super.m_type = 1; this.width
= width; } }
class Circle extends Shape { int r; Circle(int radius) { super.m_type = 1; this.r = radius; } }
```

✓ Solution

(a) It violates the **Open/Closed Principle**: GraphicEditor must be modified every time a new shape is added (another m_type branch), and it reaches into the fields of every subclass. (There is also a bug: Circle sets m_type = 1, so a circle's area is computed as a square.) Rewrite — each shape computes its own area:

```
abstract class Shape { abstract double getArea(); }
class Rectangle extends Shape {          // the paper's 'Rectangle' is a square: width * width
    private final int width;
    Rectangle(int width) { this.width = width; }
    double getArea() { return width * width; }
}
class Circle extends Shape {
    private final int r;
    Circle(int radius) { this.r = radius; }
    double getArea() { return 3.1416 * r * r; }
}
class GraphicEditor {                    // closed for modification
    double getArea(Shape s) { return s.getArea(); }
}
// new Triangle extends Shape { double getArea() {...} }  -> no change to GraphicEditor
```

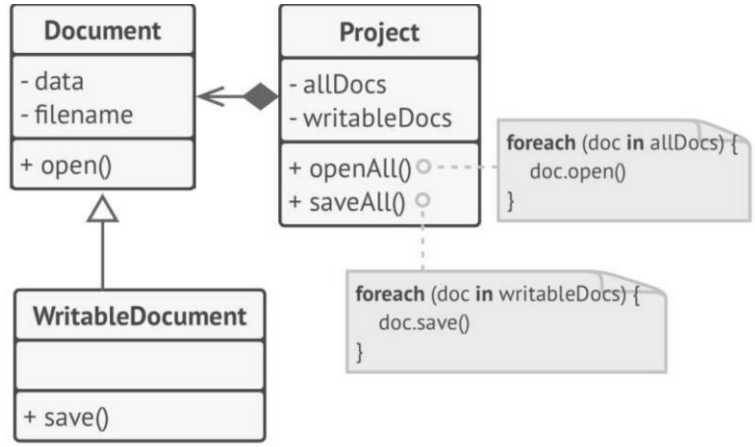
(b) SRP: a class should have only one reason to change. Violation: class Employee { calculatePay(); saveToDatabase(); printTimeSheetReport(); } — pay rules, storage and report format are three reasons. Fix: Employee (data + calculatePay), EmployeeRepository.save(e), TimeSheetReport.print(e).

Lesson 3 — LSP, ISP and DIP

Slides 18–31 (Lec 1 - Design Principle)

1. Liskov Substitution Principle (sl. 18–20)

- You must be able to pass a subclass object wherever a parent object is expected without breaking the client.
- When overriding, extend the parent's behaviour; do not replace it with something different.
- Violation (sl. 19): ReadOnlyDocument extends Document and overrides save() to throw an exception. Project.saveAll() must now check the type — the subclass is not substitutable.
- Fix (sl. 20): Document has only open(); WritableDocument extends Document and adds save(). Project keeps allDocs and writableDocs, and saves only writable ones.



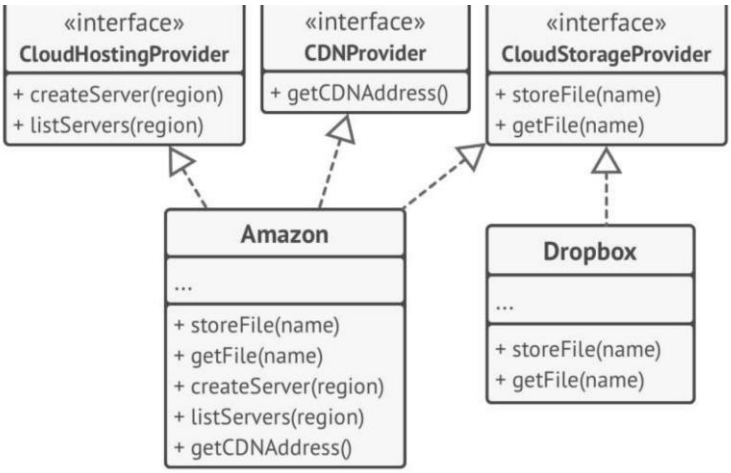
AFTER: the problem is solved after making the read-only document class the base class of the hierarchy.

Figure 3.1 — LSP fix: only WritableDocument has save() (sl. 20)

2. Interface Segregation Principle (sl. 21–22)

Clients shouldn't be forced to depend on methods they do not use. Make interfaces narrow.

Example: a fat CloudProvider interface (storeFile, getFile, createServer, listServers, getCDNAddress). Amazon can implement all, but a Dropbox adapter must leave hosting/CDN methods empty or throwing. Split into CloudHostingProvider, CDNProvider and CloudStorageProvider; Amazon implements all three, Dropbox only CloudStorageProvider.



AFTER: one bloated interface is broken down into a set of more granular interfaces.

Figure 3.2 — ISP: the fat interface split into three (sl. 22)

3. Dependency Inversion Principle (sl. 23–31)

- High-level classes shouldn't depend on low-level classes; both depend on abstractions.
- Abstractions shouldn't depend on details; details depend on abstractions.
- sl. 24–25: BudgetReport → MySQLDatabase is inverted to BudgetReport → «interface» Database ← MySQL, MongoDB.
- How it works (sl. 26): the high-level module defines the interface it needs; low-level modules implement it.

Worked example — the power switch (sl. 27–31)

```
// Without DIP: ElectricPowerSwitch can only switch a LightBulb
public class ElectricPowerSwitch { public LightBulb lightBulb; ... lightBulb.turnOn(); ... }

// With DIP
public interface Switch     { boolean isOn(); void press(); }
public interface Switchable { void turnOn(); void turnOff(); }
public class ElectricPowerSwitch implements Switch {
    public Switchable client; public boolean on;
    public ElectricPowerSwitch(Switchable client) { this.client = client; this.on = false; }
    public boolean isOn() { return on; }
    public void press() {
        if (isOn()) { client.turnOff(); on = false; } else { client.turnOn(); on = true; }
    }
}
public class Fan implements Switchable { public void turnOn()  { System.out.println("Fan: Fan
turned on..."); }
                                         public void turnOff() { System.out.println("Fan: Fan
turned off..."); } }
public class LightBulb implements Switchable { /* same with 'Bulb' messages */ }
```

TRAP DIP is not the same as 'dependency injection'. Injection (passing the Switchable in the constructor) is just a technique; the principle is that both sides depend on the Switchable abstraction.

Practice — Lesson 3

(a) Actual — 2022-23 Q2(c). (b) Actual — 2012-13 Q4(b). (c) Constructed.

(a) Give an example of the Interface Segregation Principle. (8)

(b) What is the primary motivation behind ISP? Explain ISP with a suitable 'bad example'. (14⅔)

(c) Show an LSP violation with Rectangle/Square and fix it.

✓ Solution

(a)/(b) Motivation: a fat interface couples every client to every method; implementers are forced to write dummy methods, and a change to one method forces recompiling/retesting clients that never use it. Bad example:

```
interface Worker { void work(); void eat(); void attendMeeting(); }
class Robot implements Worker {
    public void work() { ... }
    public void eat() { throw new UnsupportedOperationException(); }        // forced,
meaningless
    public void attendMeeting() { throw new UnsupportedOperationException(); }
}
// ISP: split by client need
interface Workable  { void work(); }
interface Eatable   { void eat(); }
interface Meetable  { void attendMeeting(); }
class Human implements Workable, Eatable, Meetable { ... }
class Robot implements Workable { public void work() { ... } }
```

The CloudProvider/Dropbox example of sl. 21–22 is an equally good answer.

(c) Square extends Rectangle and setWidth() also sets height. A client that does r.setWidth(5); r.setHeight(4); expects area 20 but gets 16 → Square is not substitutable. Fix: make Rectangle and Square siblings under abstract Shape { double area(); } with no setters that break invariants.

Lesson 4 — What a Design Pattern Is

Slides 1–10 (Lec 2 - Basics of Design Pattern)

1. Definition (sl. 2)

- A standard solution to a common programming problem — a design structure that achieves a particular purpose.
- A description or template of how to solve a problem, usable in many situations.
- A technique for flexible/efficient code: reduces coupling and memory overhead.
- Shorthand for describing a design (a vocabulary).

2. The Gang of Four catalogue (sl. 3–6)

Category	Concern	Patterns (23)
Creational (5)	How objects are instantiated	Factory Method, Abstract Factory, Builder, Prototype, Singleton
Structural (7)	How objects/classes are combined into larger structures	Adapter, Bridge, Composite, Decorator, Façade, Flyweight, Proxy
Behavioral (11)	How objects communicate and share responsibilities	Interpreter, Template Method, Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor

3. Essential elements of a pattern (sl. 7)

Element	What it states
Intent (name)	Design goal of the pattern; the name gives a shared vocabulary
Problem	When to apply it: the problem and its context
Solution	Elements of the design, their relationships, responsibilities and collaborations (a template, not code)
Consequences	Benefits and trade-offs of applying it

4. Why patterns are needed (sl. 8–10)

Requirements always change: users' view changes after talking to developers; developers' view of the domain changes as they learn it; the environment changes. Design should accommodate change, and patterns give code that absorbs change easily.

Reasons to study (sl. 10): reuse proven solutions; common terminology; higher-level thinking; decide whether a design is right, not just working; individual and team learning; modifiability; better alternatives even without the pattern; alternatives to large inheritance hierarchies.

Practice — Lesson 4

(a) Actual — 2011-12 Q4(c). (b) Actual — 2014-15 Q6(a).

(a) What are the four essential elements of a design pattern? Discuss briefly. (8)

(b) What is the difference between structural and behavioral patterns? Give criteria for classifying a new pattern, explaining why current patterns are classified as they are. (10)

✓ Solution

(a) Name/intent, problem, solution, consequences — table in §3, one sentence each with Observer as the example (problem: dependents must be updated on change; solution: Subject keeps a list of Observer interfaces and calls update(); consequence: loose coupling, but unexpected update cascades).

(b) Structural patterns are about **composition** — how classes and objects are assembled into larger structures while staying flexible (Adapter wraps to change an interface, Composite builds trees, Decorator wraps to add responsibilities). Behavioral patterns are about **interaction** — algorithms, control flow and assignment of responsibility between objects (Observer: who notifies whom; Strategy: which algorithm runs; Mediator: who talks to whom).

Criteria for a new pattern: (1) If its main purpose is to define the static shape of a structure — which object contains, wraps or refers to which, so that a larger object is formed — it is structural. (2) If its main purpose is to distribute a run-time responsibility or communication protocol among objects — message flow, choice of algorithm, state changes — it is behavioral. (3) If it is mainly about creating objects, it is creational. Decorator is structural because its essence is a wrapping structure with the same interface; Strategy is behavioral because its essence is selecting an algorithm at run time, even though both use composition.

Quick Notes — Design Principles

Revision points for Lessons 1–4.

L1 · Encapsulate, interfaces, composition

- Encapsulate what varies: getTaxRate() method → TaxCalculator class.
- Program to an interface: Cat.eat(Food), not eat(Sausage).
- Diamond problem → Java bans multiple class inheritance. Favour composition/delegation.
- High cohesion + low coupling; subclassing vs delegation → prefer delegation.

L2 · SRP, OCP

- SRP: one reason to change (TimeSheetReport out of Employee).
- OCP: open for extension, closed for modification. Type-code if-chains → abstract method per subclass.
- 2016-17 Q8(d): GraphicEditor violates OCP; each Shape gets getArea().

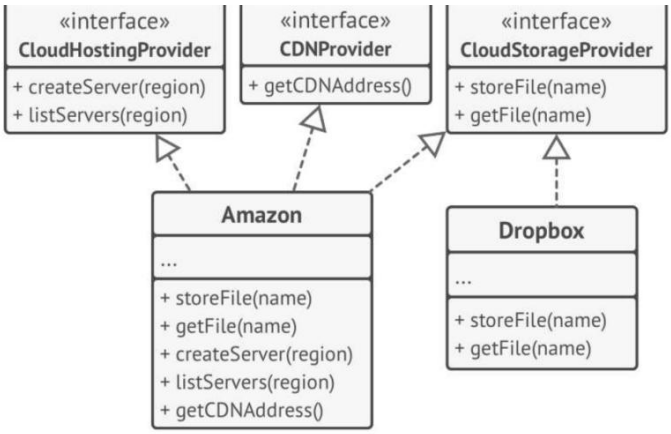
L3 · LSP, ISP, DIP

- LSP: subclass substitutable; ReadOnlyDocument.save() throwing = violation → WritableDocument.
- ISP: no fat interfaces; CloudProvider → Hosting / CDN / Storage; Worker → Workable / Eatable.
- DIP: both depend on abstraction; Switch/Switchable with Fan and LightBulb.

L4 · Pattern basics

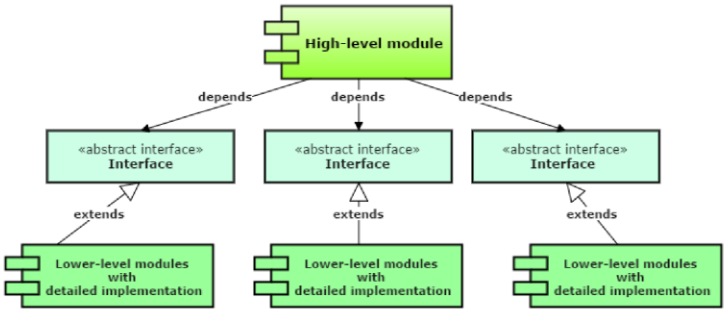
- GoF 23: creational 5, structural 7, behavioral 11.
- Elements: intent/name, problem, solution, consequences.
- Structural = composition of structure; behavioral = interaction/responsibility.

Key Diagrams to Reproduce



AFTER: one bloated interface is broken down into a set of more granular interfaces.

ISP split of CloudProvider (sl. 22)



Inverted dependency (sl. 25)